

Redux

Введение в Redux

Redux — это предсказуемый контейнер состояния для JavaScript-приложений. Он помогает писать приложения, которые ведут себя согласованно, работают в различных средах (клиент, сервер и нативные приложения) и их легко тестировать.

Redux был создан Дэном Абрамовым в 2015 году и быстро стал популярным решением для управления состоянием в React-приложениях, хотя он может использоваться с любой библиотекой или фреймворком пользовательского интерфейса.

Основные принципы Redux

Redux основан на трех фундаментальных принципах:

1. **Единственный источник истины:** Состояние всего приложения хранится в дереве объектов внутри одного хранилища (store).
2. **Состояние только для чтения:** Единственный способ изменить состояние — отправить действие (action), которое описывает, что произошло.
3. **Изменения производятся с помощью чистых функций:** Редьюсеры (reducers) — это чистые функции, которые принимают предыдущее состояние и действие, и возвращают новое состояние.

Основные концепции Redux

Store (Хранилище)

Хранилище — это объект, который содержит состояние приложения, предоставляет методы для доступа к состоянию, отправки действий и регистрации слушателей.

```
import { createStore } from 'redux';
import rootReducer from './reducers';

const store = createStore(rootReducer);
```

Actions (Действия)

Действия — это объекты, которые описывают, что произошло в приложении. Они должны иметь свойство `type`, которое указывает тип выполняемого действия.

```
// Действие
const addTodo = {
  type: 'ADD_TODO',
  text: 'Изучить Redux'
```

```
};

// Создатель действия
function addTodo(text) {
  return {
    type: 'ADD_TODO',
    text
  };
}
```

Reducers (Редьюсеры)

Редьюсеры — это чистые функции, которые принимают предыдущее состояние и действие, и возвращают новое состояние. Они определяют, как состояние приложения изменяется в ответ на действия.

```
function todosReducer(state = [], action) {
  switch (action.type) {
    case 'ADD_TODO':
      return [
        ...state,
        {
          id: Date.now(),
          text: action.text,
          completed: false
        }
      ];
    case 'TOGGLE_TODO':
      return state.map(todo =>
        todo.id === action.id ? { ...todo, completed: !todo.completed } : todo
      );
    default:
      return state;
  }
}
```

Dispatch (Отправка)

Отправка — это метод хранилища, который используется для отправки действий в хранилище.

```
store.dispatch(addTodo('Изучить Redux'));
```

Selectors (Селекторы)

Селекторы — это функции, которые извлекают части состояния для компонентов.

```
function getVisibleTodos(todos, filter) {
  switch (filter) {
    case 'SHOW_ALL':
      return todos;
    case 'SHOW_COMPLETED':
      return todos.filter(t => t.completed);
    case 'SHOW_ACTIVE':
      return todos.filter(t => !t.completed);
    default:
      throw new Error('Unknown filter: ' + filter);
  }
}
```

Использование Redux с React

Для использования Redux с React обычно используется библиотека `react-redux`, которая предоставляет компоненты и хуки для связывания React и Redux.

Установка

```
npm install redux react-redux
```

```
# или с использованием yarn
yarn add redux react-redux
```

Provider

Provider — это компонент высшего порядка, который делает хранилище Redux доступным для всех компонентов в дереве компонентов.

```
import React from 'react';
import ReactDOM from 'react-dom';
import { Provider } from 'react-redux';
import { createStore } from 'redux';
import rootReducer from './reducers';
import App from './App';

const store = createStore(rootReducer);

ReactDOM.render(
  <Provider store={store}>
    <App />
  </Provider>,
  document.getElementById('root')
);
```

useSelector

`useSelector` — это хук, который позволяет извлекать данные из состояния хранилища Redux.

```
import React from 'react';
import { useSelector } from 'react-redux';

function TodoList() {
  const todos = useSelector(state => state.todos);

  return (
    <ul>
      {todos.map(todo => (
        <li key={todo.id}>{todo.text}</li>
      ))}
    </ul>
  );
}
```

useDispatch

`useDispatch` — это хук, который возвращает функцию `dispatch` из хранилища Redux.

```
import React, { useState } from 'react';
import { useDispatch } from 'react-redux';

function AddTodo() {
  const [text, setText] = useState('');
  const dispatch = useDispatch();

  const handleSubmit = e => {
    e.preventDefault();
    if (!text.trim()) return;
    dispatch({ type: 'ADD_TODO', text });
    setText('');
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={text} onChange={e => setText(e.target.value)} />
      <button type="submit">Добавить задачу</button>
    </form>
  );
}
```

connect (устаревший подход)

До появления хуков в React, для связывания компонентов с Redux использовалась функция `connect`.

```
import { connect } from 'react-redux';

function TodoList({ todos, toggleTodo }) {
  return (
    <ul>
      {todos.map(todo => (
        <li
          key={todo.id}
          onClick={() => toggleTodo(todo.id)}
          style={{ textDecoration: todo.completed ? 'line-through' : 'none' }}
        >
          {todo.text}
        </li>
      ))}
    </ul>
  );
}

const mapStateToProps = state => ({
  todos: state.todos
});

const mapDispatchToProps = dispatch => ({
  toggleTodo: id => dispatch({ type: 'TOGGLE_TODO', id })
});

export default connect(mapStateToProps, mapDispatchToProps)(TodoList);
```

Middleware

Middleware в Redux предоставляет способ расширить функциональность хранилища. Middleware перехватывает действия перед тем, как они достигнут редьюсера, что позволяет выполнять побочные эффекты, логирование, асинхронные запросы и т.д.

Redux Thunk

Redux Thunk — это middleware, который позволяет писать создателей действий, которые возвращают функцию вместо объекта действия. Это позволяет отложить отправку действия или отправить его только при выполнении определенного условия.

```
import { createStore, applyMiddleware } from 'redux';
import thunk from 'redux-thunk';
import rootReducer from './reducers';

const store = createStore(rootReducer, applyMiddleware(thunk));

// Асинхронный создатель действия
function fetchTodos() {
```

```

return async function(dispatch) {
  dispatch({ type: 'FETCH_TODOS_REQUEST' });
  try {
    const response = await fetch('/api/todos');
    const todos = await response.json();
    dispatch({ type: 'FETCH_TODOS_SUCCESS', todos });
  } catch (error) {
    dispatch({ type: 'FETCH_TODOS_FAILURE', error });
  }
};
}

// Использование
store.dispatch(fetchTodos());

```

Redux Saga

Redux Saga — это middleware для обработки побочных эффектов в Redux. Она использует генераторы JavaScript для создания более декларативных и тестируемых побочных эффектов.

```

import { createStore, applyMiddleware } from 'redux';
import createSagaMiddleware from 'redux-saga';
import rootReducer from './reducers';
import { call, put, takeEvery } from 'redux-saga/effects';

// Saga
function* fetchTodosSaga() {
  try {
    const response = yield call(fetch, '/api/todos');
    const todos = yield response.json();
    yield put({ type: 'FETCH_TODOS_SUCCESS', todos });
  } catch (error) {
    yield put({ type: 'FETCH_TODOS_FAILURE', error });
  }
}

function* rootSaga() {
  yield takeEvery('FETCH_TODOS_REQUEST', fetchTodosSaga);
}

// Создание хранилища с middleware
const sagaMiddleware = createSagaMiddleware();
const store = createStore(rootReducer, applyMiddleware(sagaMiddleware));

// Запуск саги
sagaMiddleware.run(rootSaga);

// Использование
store.dispatch({ type: 'FETCH_TODOS_REQUEST' });

```

Redux Toolkit

Redux Toolkit — это официальный набор инструментов для эффективной разработки с Redux. Он включает утилиты для упрощения распространенных случаев использования, таких как настройка хранилища, создание редьюсеров, иммутабельное обновление и многое другое.

```
import { configureStore, createSlice } from '@reduxjs/toolkit';

// Создание слайса
const todosSlice = createSlice({
  name: 'todos',
  initialState: [],
  reducers: {
    addTodo: (state, action) => {
      state.push({
        id: Date.now(),
        text: action.payload,
        completed: false
      });
    },
    toggleTodo: (state, action) => {
      const todo = state.find(todo => todo.id === action.payload);
      if (todo) {
        todo.completed = !todo.completed;
      }
    }
  }
});

// Экспорт действий и редьюсера
export const { addTodo, toggleTodo } = todosSlice.actions;
export default todosSlice.reducer;

// Создание хранилища
const store = configureStore({
  reducer: {
    todos: todosSlice.reducer
  }
});
```

Заключение

Redux предоставляет мощный и предсказуемый способ управления состоянием в JavaScript-приложениях. Хотя он имеет некоторую кривую обучения и требует написания дополнительного кода, преимущества, которые он предоставляет в плане предсказуемости, тестируемости и отладки, делают его популярным выбором для управления состоянием в крупных приложениях.

С появлением Redux Toolkit и хуков в React, использование Redux стало проще и менее многословным, что делает его более доступным для новых разработчиков.

